02613 Python and High Performance Computing

Answers for Week 13

---

1. Execessive I/O

Consider the following Python program that prints a short story about a number for the first 100,000 integers:

```
for i in range(100_000):
    # Story by ChatGPT
    print(
        f"Once upon a time, {i} felt lonely, being just one among many. "
        f"Dreaming of distinction, {i} doubled itself, becoming 2*{i}. "
        f"Still, it wasn't enough, so {i} squared itself, turning into {i}^2. "
        f" Realizing every transformation retained its essence, "
        f"{i} learned to cherish its identity, understanding it was unique, "
        f"infinite, essential-forever {i}, the protagonist of its own "
        f"mathematical saga."
    )
```

1. Make a job script that runs the above Python program. Add the `-u` option to Python to run with unbuffered output. You must submit the job to the `hpc` queue, request 1 core and a CPU model for repeatability. Also, make sure you specify output files with the `-o`/`-e` options and that the files are under `/work3/02613/dump/`.

   Answer:

   ```
   #!/bin/bash
   #BSUB -J print
   #BSUB -q hpc
   #BSUB -W 00:10
   #BSUB -n 1
   #BSUB -R "select[model==XeonE5_2650v4]"
   #BSUB -R "rusage[mem=512MB]"
   #BSUB -o /work3/02613/dump/printlots_%J.out
   #BSUB -e /work3/02613/dump/printlots_%J.err

   # Initialize Python environment
   source /dtu/projects/02613_2024/conda/conda_init.sh
   conda activate 02613
   ```

```
14
15 python -u printlots.py
```

2. Make another job script where you manually redirect the output of the Python program to a separate set of files. Again, make sure these files are also under `/work3/02613/dump/`. Hint: see the slides.

Answer: Assuming one has access to a directory under `/work3`:

```
 1 #!/bin/bash
 2 #BSUB -J print
 3 #BSUB -q hpc
 4 #BSUB -W 00:10
 5 #BSUB -n 1
 6 #BSUB -R "select[model==XeonE5_2650v4]"
 7 #BSUB -R "rusage[mem=512MB]"
 8 #BSUB -o /work3/02613/dump/printlots_%J.out
 9 #BSUB -e /work3/02613/dump/printlots_%J.err
10
11 # Initialize Python environment
12 source /dtu/projects/02613_2024/conda/conda_init.sh
13 conda activate 02613
14
15 python -u printlots.py \
16     1> /work3/02613/dump/output_${LSB_JOBID}.txt \
17     2> /work3/02613/dump/error_${LSB_JOBID}.txt
```

3. Submit both jobs. Check the job resource summaries at the end of the output files. Did the output get redirected as expected? What is the run time difference?

Answer: The resource summary for the job *without* manual output redirection was:

```
 1 ------------------------------------------------------------
 2
 3 Successfully completed.
 4
 5 Resource usage summary:
 6
 7     CPU time :                                   7.37 sec.
 8     Max Memory :                                 6 MB
 9     Average Memory :                             4.00 MB
10     Total Requested Memory :                     512.00 MB
11     Delta Memory :                               506.00 MB
```

```
12      Max Swap :                                   -
13      Max Processes :                              4
14      Max Threads :                                5
15      Run time :                                   80 sec.
16      Turnaround time :                            78 sec.
17
18  The output (if any) is above this job summary.
```

The resource summary for the job *with* manual out redirection was:

```
 1  ------------------------------------------------------------
 2
 3  Successfully completed.
 4
 5  Resource usage summary:
 6
 7      CPU time :                                   1.48 sec.
 8      Max Memory :                                 -
 9      Average Memory :                             -
10      Total Requested Memory :                     512.00 MB
11      Delta Memory :                               -
12      Max Swap :                                   -
13      Max Processes :                              -
14      Max Threads :                                -
15      Run time :                                   3 sec.
16      Turnaround time :                            4 sec.
17
18  The output (if any) is above this job summary.
```

From the `Run time :` lines, we can see that without manual redirection, the job took 80 seconds. With manual redirection, where we don't rely on the `-o`/`-e` channels, the run time decreased to only 3 seconds. That is 26 times faster!

---

2. Multi-Threaded NumPy

Consider the following Python program, which creates two sets of 100 random 1000 x 1000 matrices and then multiplies them together. At the end, it prints out the time it took.

```
1  from time import perf_counter as time
2  import numpy as np
3
4  def matmuls(A, B):
5      assert A.shape[0] == B.shape[0], "A and B must hold same number of
           matrices"
6      assert A.shape[2] == B.shape[1], "A and B must hold compatible
           matrices"
```

```
 7      n = A.shape[0]
 8      C = np.empty((n, A.shape[1], B.shape[2]))
 9      for i in range(n):
10          C[i] = np.matmul(A[i], B[i])
11      return C
12
13  A = np.random.rand(100, 1000, 1000)
14  B = np.random.rand(100, 1000, 1000)
15  t0 = time()
16  C = matmuls(A, B)
17  t1 = time()
18  print(t1 - t0)
```

1. Write a job script that runs the Python program. You must submit the job to the `hpc` queue, request 1 core and a CPU model for repeatability. Submit the job. What is the run time?

   Answer:

   ```
    1  #!/bin/bash
    2  #BSUB -J matmuls
    3  #BSUB -q hpc
    4  #BSUB -W 00:10
    5  #BSUB -n 1
    6  #BSUB -R "span[hosts=1]"
    7  #BSUB -R "select[model==XeonE5_2650v4]"
    8  #BSUB -R "rusage[mem=4GB]"
    9  #BSUB -o batch_output/matmuls_%J.out
   10  #BSUB -e batch_output/matmuls_%J.err
   11
   12  # Initialize Python environment
   13  source /dtu/projects/02613_2024/conda/conda_init.sh
   14  conda activate 02613
   15
   16  python -u matmuls.py
   ```

   The exact run time will vary depending on the chosen CPU model. With the above job script, I got 5.87 seconds.

   ---

2. Change the number of cores to 8 cores and submit again. Did the run time change?

   Answer: Change the `\#BSUB -n 1` line to `\#BSUB -n 8`. With this, my run time was 5.96 seconds, so there is effectively no change. NumPy has *not* automatically taken advantage of the extra available cores.

   ---

3. **Autolab** Change the job script to enable NumPy multi threading with 8 threads (one for each requested core). Submit again. Did the run time change?
   Hint: see the slides.

Answer: Autolab exercise - Job script omitted.

Telling NumPy (or, rather, the NumPy backend) to use 8 threads decreased the run time to 1.28 seconds - which is 4.6 times faster. Not quite a perfect speed-up, but still good!

---

4. Change the Python program such that the `matmuls` function is parallelized with multi-threading using a `ThreadPool` from the `multiprocessing` module. Submit again. What is the run time now? Why can we use multi-threading for this program instead of using multi-*processing*?
   Hint: see the slides from week 5.

Answer: A simple way is to use the `starmap` method of the `ThreadPool` class to parallelize the for loop in the `matmuls` function. The Python program will then be:

```
 1  from time import perf_counter as time
 2  import numpy as np
 3  from multiprocessing.pool import ThreadPool
 4
 5  def matmuls(A, B):
 6      assert A.shape[0] == B.shape[0], "A and B must hold same
           number of mats"
 7      assert A.shape[2] == B.shape[1], "A and B must hold compatible
            matrices"
 8      n = A.shape[0]
 9      with ThreadPool(8) as p:
10          C = np.concatenate(p.starmap(np.matmul, zip(A, B)))
11      return C
12
13  A = np.random.rand(100, 1000, 1000)
14  B = np.random.rand(100, 1000, 1000)
15  t0 = time()
16  C = matmuls(A, B)
17  t1 = time()
18  print(t1 - t0)
```

Running the job with the modified Python program now gives a run time of 1.87 seconds. It is now slower than before, since we are using multi-threading with NumPy *and* with a thread pool. Now, each thread in the pool also tries to run a multi-threaded matrix multiplication, which means we are running many more threads than we have cores available. This means a lot of time is wasted on scheduling the threads and we end up running slower overall.

The reason we can make use of a thread pool instead of a process pool is because NumPy releases the GIL. Since most of the time is spent on running the matrix multiplications, each thread spends most of its time with the GIL released.

---

5. Change your job script to disable NumPy multi-threading (i.e., remove what you added in exercise 2.3). Submit the job one last time. What is the run time now?

Answer: With the NumPy multi-threading disabled again (but still using the thread pool), the time is now back down to 1.33 seconds - more or less the same as with multi-threaded NumPy.

The moral of this exercise is to be careful when you are writing parallel code - whether it be multi processing or multi threading. While *you* may not be creating too many threads/processes, you are maybe using a package which *is*. And, as a result, you are leaving free performance on the table. So, check what your packages are doing!
